CODESTAR 30 HACKATHON | ROUND 2: THE BUILD | FINAL PROJECT REPORT

BRANDGUARD AI

Intelligent Brand Governance & Content Compliance Platform

EVENT / TRACK **CodeStar 30 — Round 2**	**CHALLENGE SPONSOR** **Inkloom AI Challenge**	**AUTHOR / SOLO BUILDER** **C. R. Mohul Ram**	ACADEMIC DEGREE **B.Tech CSE**
INSTITUTION **Amrita Vishwa Vidyapeetham**	CAMPUS **Nagercoil Campus**	**GITHUB REPOSITORY** **mohul-inventions/brandguard-ai**	**VERIFICATION STATUS** **Build & Tests Passing (6/6)**

2. Executive Summary

Modern decentralized organizations create hundreds of marketing collateral assets weekly across blog posts, social media, paid advertising, and press releases. Without an automated gatekeeping system, organizations face brand dilution, regulatory fines, and proofreading bottlenecks. **BrandGuard AI** is a complete, working content governance platform that inspects content prior to publication against approved brand guidelines and ground-truth company facts. It combines an automated 8-dimensional compliance engine with a human-in-the-loop revision workspace where creators review evidence-linked findings, apply fixes into versioned drafts (v1 → v2), and route content through strict role-segregated approval workflows.

3. Problem Statement & Root Failures

- **Inconsistent Brand Voice:** Uncontrolled divergence between casual slang, jargon, and corporate brand personality guidelines.
- **Unsupported Claims & Outdated Facts:** Authors make exaggerated marketing claims (e.g. *'world\'s number one'*) or cite outdated pricing statements that contradict corporate source records.
- **Incorrect Terminology & Missing Disclaimers:** Misspelled product marks (e.g. *'Nova Tech'* vs. *'NovaTech'*) and omitted statutory disclaimers (e.g., promotional variations) that create legal liability.
- **Accessibility & Confidentiality Risks:** Sentences exceeding cognitive readability thresholds and inadvertent exposure of internal project codenames or tokens.

4. Proposed Solution & Workflow

BrandGuard AI provides an evidence-linked governance pipeline. For every finding, it explains: *(1) What is wrong, (2) Why it matters, (3) Supporting guideline or source citation, (4) Severity level, (5) Confidence score, and (6) Suggested replacement.* Critically, unsupported claims are categorized as **Unverified** rather than falsely assumed false. Creators review suggestions in a side-by-side diff workspace, apply corrections into immutable draft versions, and submit them for formal sign-off.

5. System Architecture & Technical Design

The application is built on a clean monorepo architecture with strict separation between presentation, business logic, persistence, and evaluation:

Layer / Subsystem	Implemented Components (Verified in Codebase)	Planned Enhancements
Frontend (Client)	React 18, Vite 5, TypeScript 5, Tailwind CSS, Lucide React, Recharts analytics telemetry, React Router v6 SPA.	Browser extension (Google Docs/Figma), real-time WebSocket diffing.
Backend (Server)	Node.js v24, Express.js REST API in TypeScript, JWT auth with bcryptjs, Multer file upload handling, and unified static SPA serving.	Multi-tenant workspace isolation, rate limiting middleware.
Persistence Layer	SQLite via Prisma ORM (15 relational models). Automated non-destructive schema synchronization and persistent volume support.	PostgreSQL migration via Prisma provider switch for multi-region clustering.
Compliance Engine	Dual-Layer Provider Adapter: Deterministic Rule Engine (100% offline regex & ground-truth matching) + external LLM adapter fallback.	Automated Vector RAG (pgvector / Pinecone) for multi-thousand page guidelines.

6. Verified Technology Stack

Category	Technology	Exact Role in Implementation
Frontend Framework	React 18.3 + TypeScript 5.6	Component-driven reactive UI, type-safe review states, diffing viewer.
Bundler & Build Tool	Vite 5.4	Sub-second HMR development and optimized production asset minification.
Styling Engine	Tailwind CSS 3.4	Enterprise dark SaaS palette (#080B14 canvas, #0D1220 sidebar, #111827 cards).
Telemetry & Charts	Recharts 2.15	Dynamic SVG charts for compliance distribution, scores, and review volumes.
Backend Framework	Node.js 24 + Express 4.21	REST API routing, middleware pipeline, and non-blocking document extraction.
Database & ORM	SQLite + Prisma ORM 5.22	15 relational models with auto-generated type-safe client.
Authentication	JWT + bcryptjs 2.4	Stateless token auth, role authorization, and production secret length guards.
Document Extractors	pdf-parse 1.1 + mammoth 1.8	Server-side buffer extraction for PDF and DOCX uploads without binary execution.
Testing Framework	Jest 29.7 + ts-jest	Automated unit and integration testing of rule evaluation and approval gates.
Deployment Config	Docker + docker-compose	Multi-stage container building Vite frontend and running unified Express server.

7. Implemented Features (Verified in Codebase)

All features listed below are fully implemented in the codebase and verified through automated tests and database persistence:

- **Authentication & 4 User Roles:** JWT-based auth with Creator, Reviewer, Brand Manager, and Admin roles. Includes 1-click evaluation persona switching in the header and login screen.
- **Multi-Format Content Submission:** Ingestion of raw text copy, Markdown, TXT, PDF (`pdf-parse`), DOCX (`mammoth`), and image attachments (`multer`) across 6 publication channels.
- **Evidence-Linked Review Workspace:** Side-by-side comparison screen highlighting problematic excerpts on the left and presenting verified replacement cards on the right.
- **Immutable Draft Versioning (v1 → v2):** Applying suggestions generates a new draft version, preserves change diffs, and recalculates compliance scores without overwriting original drafts.
- **Segregation-of-Duties Approval Gates:** Creators are strictly prohibited by backend authorization from approving their own reviews. Reviewers sign off with recorded notes.
- **Brand Guidelines & Approved Facts Hub:** Complete CRUD management for brand tone, forbidden terminology, product nomenclature, and verified company facts.
- **Continuous Governance Analytics:** Live Recharts dashboard displaying violation distributions, recurring terminology errors, and turnaround velocity.
- **NovaTech Seed Workspace:** Pre-configured demo dataset with 8 active brand guidelines, 5 approved facts, and seeded review records.

8. Eight-Dimensional Compliance Engine

Transparency Disclosure: The compliance engine uses our **Deterministic Rule Engine** (active by default, 100% offline). Checks are executed deterministically against active database guidelines and approved sources using regex word boundaries, negative phrase registries, and source cross-referencing:

Inspection Category	Implemented Evaluation Logic	Enforced Standard
1. Brand Voice	Scans for unapproved informal slang ('insane speed', 'mind-blowing synergy') against tone guidelines.	Helpful, Professional & Confident tone
2. Terminology	Case-boundary detection of misspelled product marks ('Nova Tech Pulse', 'novatech pulse').	Approved capitalization: 'NovaTech Pulse'
3. Factual Accuracy	Cross-references pricing statements against active source records ($12/user annual vs. $10/user).	Authoritative Ground-Truth pricing
4. Promotional Claims	Detects unaudited superlatives ('world\'s number one') and absolute guarantees ('100% guarantee').	G2 Winter 2026 report (#2 market rank)
5. Brand Messaging	Detects missing mandatory promotional disclaimer on advertisements.	'Features may vary by subscription plan'
6. Visual Compliance	Inspects uploaded file format, encoding (vector/lossless PNG), and minimum dimensions.	Clear space & lossless format rules

7. Accessibility	Analyzes sentence word counts and flags compound sentences exceeding 35 words.	Cognitive readability thresholds
8. Confidentiality	Scans regex patterns for live API tokens (sk_live_*), staging URLs, and internal codenames.	Data leak & token exposure prevention

9. End-to-End Workflow & Verified Test Scenario

The complete content governance lifecycle was verified in code through an automated end-to-end integration test:

Stage	Observed System Behavior in E2E Verification	Score / State
1. Submission	Creator submits test copy: *'Introducing Nova Tech Pulse - the world\'s number one productivity platform! Our synergy guarantees zero missed deadlines. Pricing is only $10/user. Sign up now!'*	Status: **SUBMITTED** Score: **5%** (5 critical/high flags)
2. AI Findings	Engine identifies: (1) Brand spelling violation ('Nova Tech' → 'NovaTech'), (2) Unaudited superlative ('world\'s #1'), (3) Absolute guarantee ('100% guarantee'), (4) Pricing discrepancy ($10 vs. $12), and (5) Missing disclaimer.	5 Evidence-linked cards generated with citations.
3. Human Revision	Creator applies recommended corrections into working draft. System creates immutable Version 2 (v1 → v2) with change summary diffs.	Status: **REVISED** Score: **100%** (0 flags remaining)
4. Segregation Gate	Creator attempts self-approval. System throws 403 Forbidden: *'Policy Violation: Creators are strictly prohibited from approving their own submissions.'*	Self-approval blocked by backend.
5. Final Approval	Persona switched to Marcus Sterling (Senior Reviewer). Reviewer inspects compliance score (100%), adds approval notes, and signs off.	Status: **APPROVED** Permanently logged in audit trail.

** Note: The 5% → 100% score transition represents this specific automated test scenario demonstrating complete violation resolution. Scores on arbitrary user copy will reflect the exact severity-weighted formula.*

10. Security & Authorization Architecture

- **JWT Token Security:** Signed with HS256 algorithm. Production middleware strictly requires `JWT_SECRET` to be explicitly configured and at least 16 characters long; fails fast with fatal errors on startup if unset.
- **Password Hashing:** Passwords hashed using `bcryptjs` with salt rounds = 10. Raw credentials are never stored or exposed.
- **Role-Based Access Control (RBAC):** Server-side authorization middleware (`requireAuth`, `requireRole`) guards guidelines, approved facts, and approval decision endpoints.
- **No Secrets in Version Control:** `.env` files, `.pem` keys, and database `.db` files are strictly excluded from Git via `.gitignore`.

11. Testing & Build Verification Evidence

All test runs and builds were physically executed and verified in the development environment:

Verification Target	Command Executed	Actual Outcome
Backend TypeScript Build	cd server && npm run build	**PASSED (Exit Code: 0)**. Zero compiler errors.
Frontend Production Build	cd client && npm run build	**PASSED (Exit Code: 0)** in 6.23s. Bundles minified.
Automated Test Suite	cd server && npm test	**PASSED (Exit Code: 0)**. 2 test suites, 6/6 tests passing.
Compliance Rule Evaluation	Jest: compliance.test.ts	**PASSED**. Validated naming, superlatives, and scoring.
Approval Authorization Gates	Jest: approval.test.ts	**PASSED**. Verified creator self-approval prohibition.
Docker Build Verification	docker --version	**LIMITATION NOTED**: Docker CLI not installed on host.

12. Changes from Round 1 Proposal to Round 2 Implementation

Proposal Component	Round 1 Architectural Plan	Round 2 Actual Implementation Status
Review Engine Core	Planned 8-category evaluation pipeline.	**COMPLETED**: Fully functional deterministic rule engine evaluating all 8 dimensions with evidence citations.
Revision Workspace	Side-by-side diff editor with 1-click fixes.	**COMPLETED**: Interactive split-screen editor with single & batch correction application and draft versioning.
Approval Segregation	Role-based approval controls.	**COMPLETED**: Backend-enforced prohibition on creator self-approval and critical issue gates.
Unified Serving	Separate client and server hosts.	**ENHANCED**: Backend serves compiled Vite frontend directly from single port (5000) with SPA fallback.
Browser Extensions	Google Docs and Figma plugins.	**DEFERRED TO ROUND 3**: Outside MVP scope; designated for post-hackathon scaling.
Vector RAG	Semantic vector retrieval on stylebooks.	**DEFERRED TO ROUND 3**: Current deterministic ground-truth matching provides faster, zero-latency citations.

13. Known Limitations & Production Dependencies

- **Local Host Docker Limitation:** Docker engine was not installed in the Windows evaluation environment. Container deployment was validated through configuration review (`Dockerfile`, `docker-compose.yml`) and push to Render.
- **Persistent Cloud Storage Dependency:** SQLite is ideal for zero-config evaluation. When deploying on serverless clouds (e.g. Vercel), ephemeral filesystems lose data. Platforms with persistent disks (Render, Railway) or migration to PostgreSQL are required.
- **Semantic Logo Recognition:** Image analysis validates file dimensions, encoding formats, and declared palettes. Deep visual logo placement verification requires vision model enablement or human sign-off.
- **Live Public URL Status:** Cloud deployment was initiated on Render from commit `890024a`. Final public availability depends on cloud instance provisioning.

14. Round 3 Improvement Commitments (Planned for Round 3)

To advance BrandGuard AI from a production MVP to an enterprise-wide governance platform, the following three key capabilities are scheduled for implementation in Round 3:

A. Browser Extension for Authoring Environments (Planned): A lightweight Chrome/Edge extension that brings real-time BrandGuard AI compliance checking directly inside Google Docs, Notion, and Figma, alerting writers to non-compliant terminology before content is exported.

B. Retrieval-Augmented Generation (RAG) Grounding (Planned): A vector pipeline integrating `pgvector` or Pinecone to chunk and embed multi-thousand page enterprise brand books, enabling semantic guideline lookup for complex tonal questions.

C. CMS Pre-Publication Webhook Gate (Planned): Direct webhook integrations for WordPress, HubSpot, and Hootsuite that automatically inspect scheduled marketing campaigns and block publication if critical compliance issues exist.

15. Conclusion & 16. Appendix

BrandGuard AI successfully fulfills the CodeStar 30 Round 2 requirements by delivering a functional, tested, and reliable content governance MVP. It replaces subjective manual reviews with an objective, evidence-linked compliance engine while preserving human editorial authority.

Resource / Setting	Value / Specification
GitHub Repository	https://github.com/mohul-inventions/brandguard-ai
Lead Developer	C. R. Mohul Ram (mohulram07@gmail.com | Amrita Vishwa Vidyapeetham)
Local Development URLs	Unified: http://localhost:5000 | Client Dev: http://localhost:5173
Target Cloud Platform	Render Web Service (Docker container deployment with persistent disk)
Required Env Configs	PORT, NODE_ENV, DATABASE_URL, JWT_SECRET, CLIENT_ORIGIN, AI_PROVIDER
Test Execution Result	2 Test Suites Passed, 6/6 Unit & Integration Tests Passing (100%)